

Sprawozdanie z projektu: Kwantyzacja wag sieci neuronowej

Wykonawcy:
Radosław Ciepał
Krzysztof Kowalik

Prowadzący: dr hab. inż. Maciej Wielgosz

13 czerwca 2026

Spis treści

1. Wstęp i opis zadania	3
2. Analizowane modele	3
2.1. Multi-layer Perceptron – wersja mniejsza	3
2.2. Multi-layer Perceptron – wersja większa	3
2.3. Convolutional Neural Network	3
2.4. Regressor	4
3. Dane wykorzystane do treningu	4
3.1. Wine recognition dataset	4
3.2. MNIST	4
3.3. CIFAR10	5
3.4. Dane poddawane regresji	5
4. Trening i testy	6
4.1. Konfiguracja procesu uczenia	6
4.2. Zastosowane metody kwantyzacji	6
4.3. Proces ewaluacji i testowania	6
5. Wyniki testów	7
5.1. Rozmiary sieci	7
5.1.1. Mały MLP	7
5.1.2. Duży MLP	7
5.1.3. CNN	8
5.1.4. Regressor	8
5.2. Osiągi sieci w zależności od stopnia kwantyzacji	8
5.2.1. Mały MLP	9
5.2.2. Duży MLP	10
5.2.3. CNN	11
5.2.4. Regressor	12
6. Wnioski	12

1. Wstęp i opis zadania

Celem tego projektu było porównanie osiągnięć oraz rozmiarów wybranych sieci neuronowych w zależności od stopnia kwantyzacji wag. Wybrane do analizy modele to Multi-layer Perceptron w dwóch rozmiarach, sieć konwolucyjna oraz sieć regresyjna. Analizowane stopnie kwantyzacji to FLOAT16 oraz INT8. Mierzone wielkości to rozmiar sieci zapisanej w pamięci, oraz metryki jakości działania danej sieci: Accuracy, Precision, Recall oraz F1 score.

2. Analizowane modele

2.1. Multi-layer Perceptron – wersja mniejsza

Ten model składa się z dwóch warstw. Pierwsza warstwa ma 13 wejść i 16 wyjść z funkcją aktywacji ReLU, druga warstwa ma 16 wejść i 3 wyjścia. Ta sieć jest bardzo mała. W praktycznych zastosowaniach sieci neuronowych, tak mały rozmiar raczej nie występuje, a zysk pamięciowy kwantyzacji jest marginalny. Jednak ten model został uwzględniony w analizie aby zaobserwować wpływ kwantyzacji na najprostsze struktury.

2.2. Multi-layer Perceptron – wersja większa

Ten model składa się z trzech warstw. Sieć przyjmuje 784 wartości, w pierwszej warstwie ta liczba jest redukowana do 512 wartości. Druga warstwa przyjmuje i generuje 512 wartości, a ostatnia, trzecia warstwa redukuje 512 wartości do wyjściowych 10 wartości. Użyta w sieci funkcja aktywacji to ReLU. Sieć tego rozmiaru zdecydowanie mogłaby być z powodzeniem zastosowana w praktyce, a jej kwantyzacja zdecydowanie jest bardziej uzasadniona względem wersji mniejszej.

2.3. Convolutional Neural Network

Ta sieć składa się z dwóch warstw konwolucyjnych i dwóch warstw fully-connected. Warstwy konwolucyjne generują kolejno 16 i 32 kanały, każda z nich operuje filtrami o wymiarach 3x3. Funkcja aktywacji użyta w tych jak i późniejszych warstwach to ReLU. Po każdej warstwie konwolucyjnej jest zastosowany pooling 2x2 czterokrotnie zmniejszający rozmiar danych przechodzący do kolejnej warstwy. Warstwy liniowe redukują liczbę wartości z 2048 do 1024, a następnie do wyjściowych 10 wartości. Ze względu na rozmiar sieci, zastosowany jest w niej dropout. Ten model jest zdecydowanie ważnym modelem w tej analizie, ze względu na swój rozmiar.

2.4. Regressor

Ostatni analizowany model, to sieć mająca za zadanie dokonać regresji. Sieć składa się z trzech warstw. Pierwsza warstwa rozszerza wejście o jednej wartości na 128 neuronów, druga warstwa redukuje do 64, a ostatnia warstwa redukuje z powrotem do pojedynczej wartości. Zawarcie tej sieci w analizie jest ważne przede wszystkim dlatego, że wszystkie pozostałe modele są modelami klasyfikującymi. Regresor pozwoli zobaczyć jak kwantyzacja wpływa na inny rodzaj sieci.

3. Dane wykorzystane do treningu

3.1. Wine recognition dataset

Ten znany zbiór danych został wykorzystany do treningu pierwszego z modeli – mniejszej wersji wielowarstwowego perceptronu. Zbiór zawiera 178 elementów, spośród których każdy posiada 13 cech, oraz jest przypisany do jednej z trzech klas. Liczba elementów jest niewielka, ale przy bardzo małej sieci nie sprawia to problemów z przeuczeniem. Przed przystąpieniem do treningu dane zostały przeskalowane za pomocą StandardScaler z biblioteki scikit-learn.

3.2 MNIST

Dla większej sieci wielowarstwowej został wybrany zbiór odręcznie pisanych cyfr MNIST. Ten zbiór składa się z 70 tysięcy czarno-białych obrazków 28x28 pikseli z ręcznie zapisanymi cyframi. Aby uczyć zwykłą sieć wielowarstwową na zbiorze obrazków, muszą one zostać najpierw przekształcone do jednowymiarowej postaci. W ten sposób obrazek o wymiarach 28x28 zamienia się w 784 wartości podawane na wejście sieci neuronowej. W tym celu wykorzystane zostały narzędzia torchvision.transforms.

3.3 CIFAR10

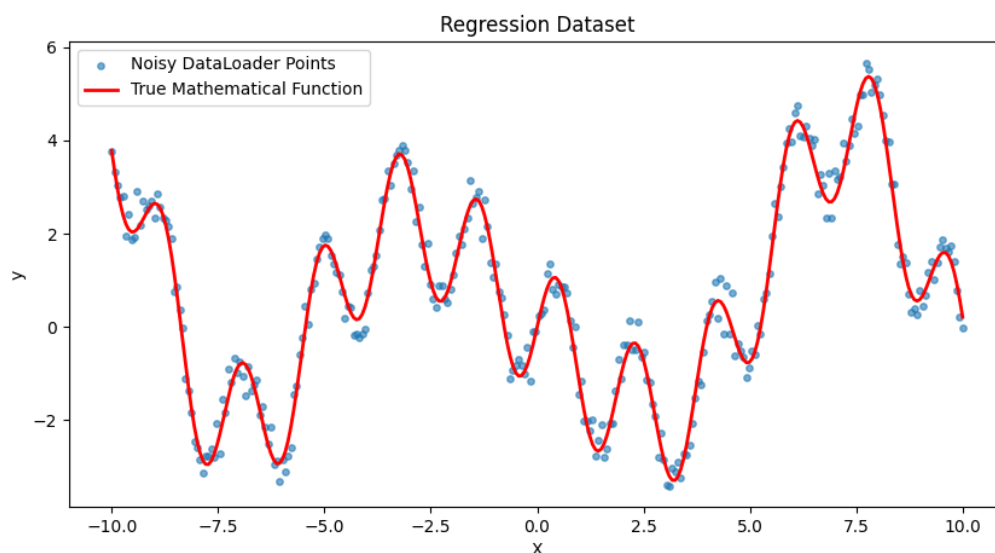
Trzeci zbiór danych, wykorzystany do treningu sieci konwolucyjnej, to równie znany i typowy zbiór CIFAR10. Zawiera on 60 tysięcy kolorowych obrazków o wymiarach 32x32 piksele, podzielonych na 10 kategorii. W tym przypadku do transformacji również użyto `torchvision.transforms`, jednak tym razem nie do redukcji wymiarów, a do normalizacji każdego z trzech kanałów.



3.4 Dane poddawane regresji

Aby pozyskać dane treningowe dla regresora, wybrana została następująca funkcja matematyczna:

$$f(x) = \sin(x) + 1.45 \cdot \sin(3.4 \cdot x) - 2.65 \cdot \sin\left(\frac{x}{1.4}\right) + 0.02 \cdot x^2$$



Dla x w przedziale $(-10, 10)$ zostało wyznaczone 300 punktów, oraz dodano losowy szum tak, jak jest to widoczne na obrazku.

4. Trening i testy

4.1. Konfiguracja procesu uczenia

Trening modeli został zrealizowany przy użyciu frameworka PyTorch. Parametry uczenia zostały dostosowane do charakterystyki rozwiązywanego problemu:

- **Modele klasyfikacyjne (MLP mały i duży, CNN)** Trenowano je przez 10 epok. Użyto optymalizatora Adam ze współczynnikiem uczenia na poziomie 0.001. Funkcją straty wykorzystywaną podczas treningu była *Cross Entropy Loss*.
- **Model regresyjny** z uwagi na specyfikę problemu trenowano znacznie dłużej – przez 150 epok. Optymalizator Adam działał z wyższym współczynnikiem uczenia równym 0.01, a jako funkcję straty wykorzystano błąd średniokwadratowy.

4.2. Zastosowane metody kwantyzacji

W ramach projektu skupiono się głównie na metodzie kwantyzacji po treningu (PTQ – *Post-Training Quantization*). Eksperymenty przeprowadzono dla następujących wariantów:

1. **Baseline (FP32):** Bazowe modele działające na 32-bitowych liczbach zmiennoprzecinkowych, użyte jako punkt odniesienia.
2. **PTQ FP16:** Konwersja modeli do 16-bitowych liczb zmiennoprzecinkowych (*Half Precision*).
3. **PTQ Dynamic INT8:** Dynamiczna kwantyzacja do 8-bitowych liczb całkowitych. Zastosowano ją głównie dla warstw liniowych (nn.Linear). W trybie dynamicznym wagi są z góry skwantyzowane, natomiast aktywacje są kwantyzowane "w locie" podczas inferencji.

4.3. Proces ewaluacji i testowania

Testowanie modeli na zbiorach walidacyjnych pozwoliło na zebranie dokładnych statystyk wydajności. Do weryfikacji skuteczności klasyfikatorów posłużono się pakietem scikit-learn, mierząc następujące metryki:

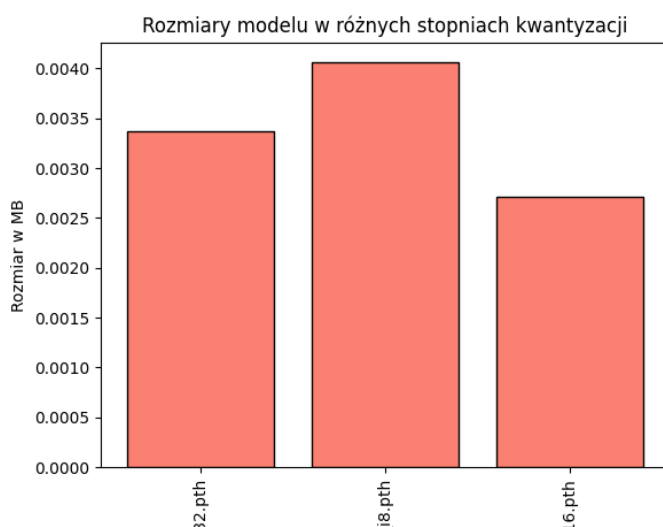
- **Accuracy:** Dokładność ogólna klasyfikacji.
- **Precision, Recall, F1-Score:** Metryki precyzji, czułości i średniej harmonicznej (F1), które policzono z wykorzystaniem uśredniania *macro* (każda klasa traktowana jest równo, niezależnie od liczności, z domyślnym wynikiem 0 w przypadku braku predykcji danej klasy).

5. Wyniki testów

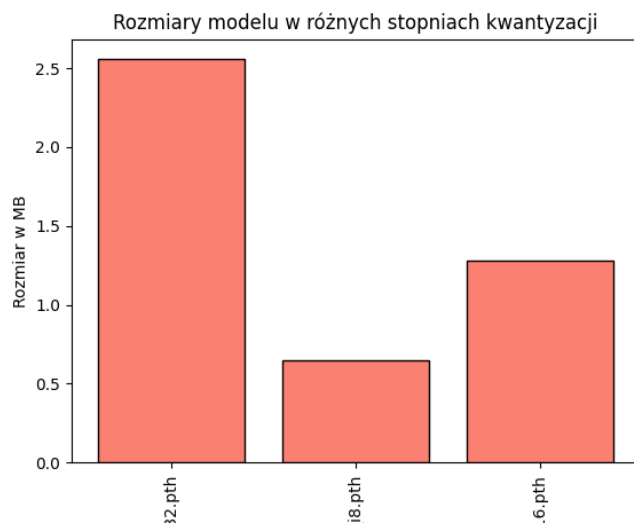
5.1. Rozmiary sieci

W przypadku większych modeli (CNN, duży MLP, Regressor) zachowanie jest zgodne z przewidywanym. Zmniejszenie rozmiaru pojedynczej wagi przekłada się w przybliżeniu na odpowiednie zmniejszenie rozmiaru całej sieci w pamięci. Jednak dla najmniejszego modelu (mały MLP) wynik jest zaskakujący: rozmiar sieci po kwantyzacji do INT8 jest większy niż oryginalny model w FLOAT32. Przyczyną mogą być dodatkowe metadane potrzebne przy kwantyzacji do typu INT8, oraz memory alignment.

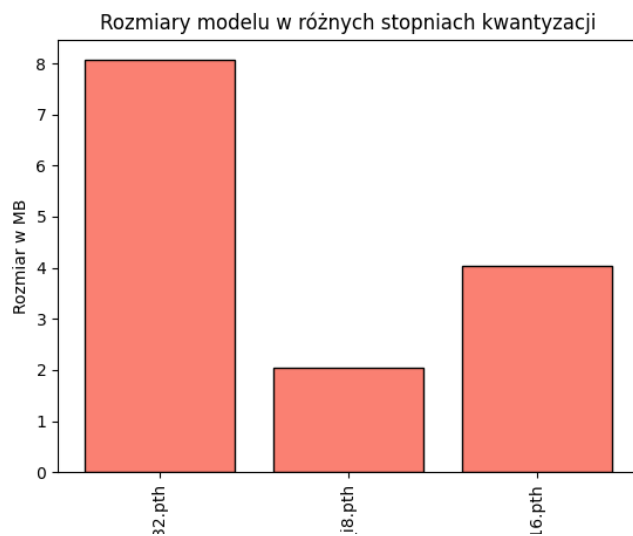
5.1.1. Mały MLP



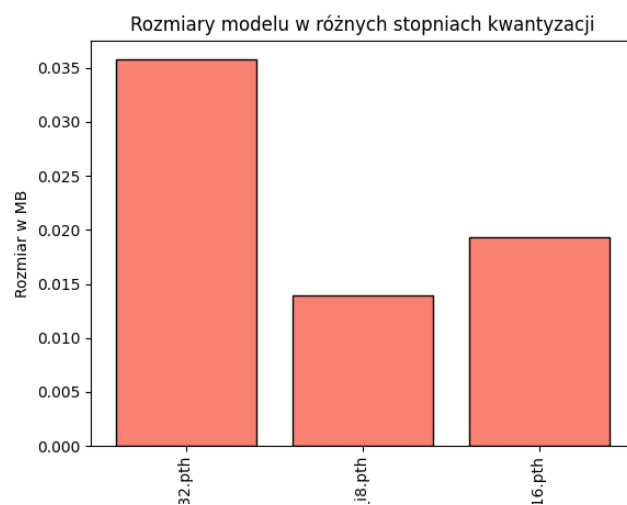
5.1.2. Duży MLP



5.1.3. CNN



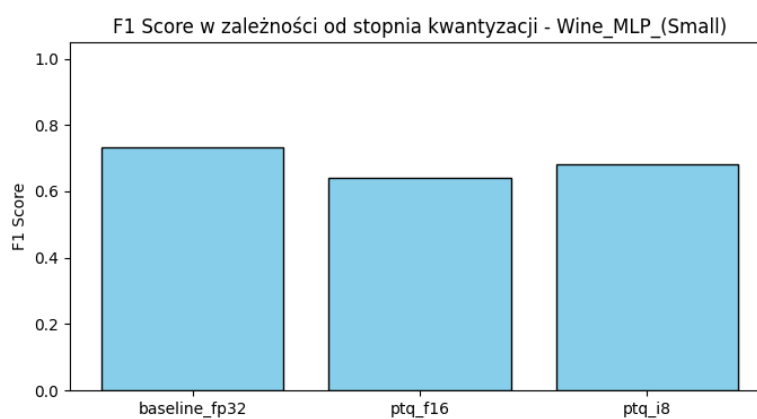
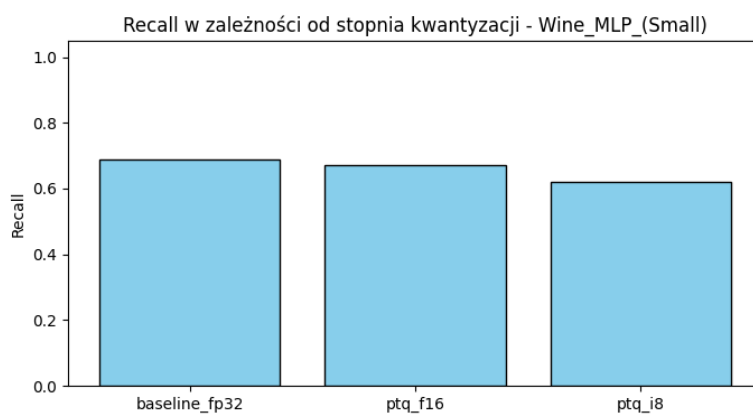
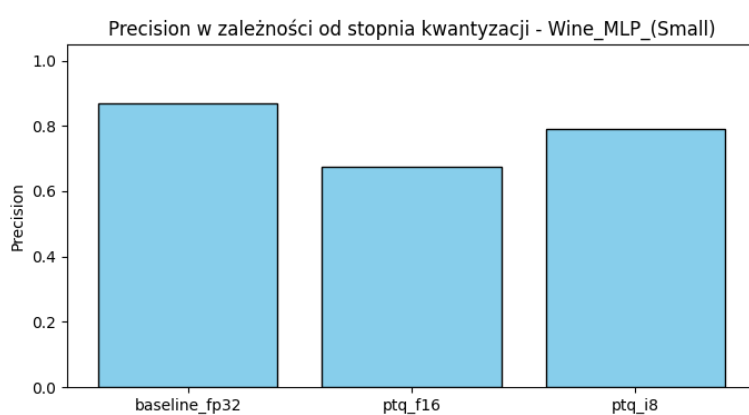
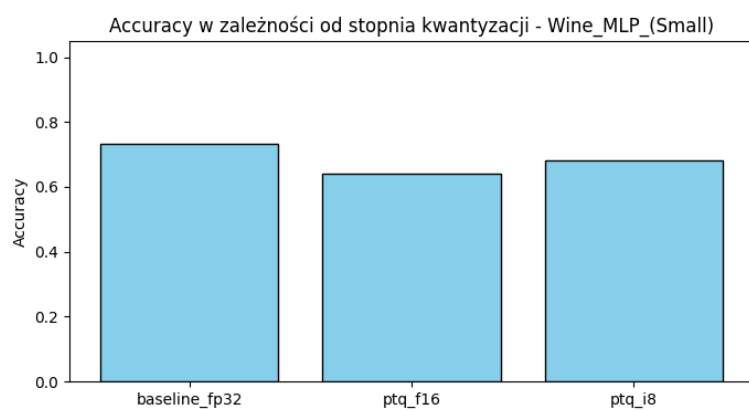
5.1.4. Regressor



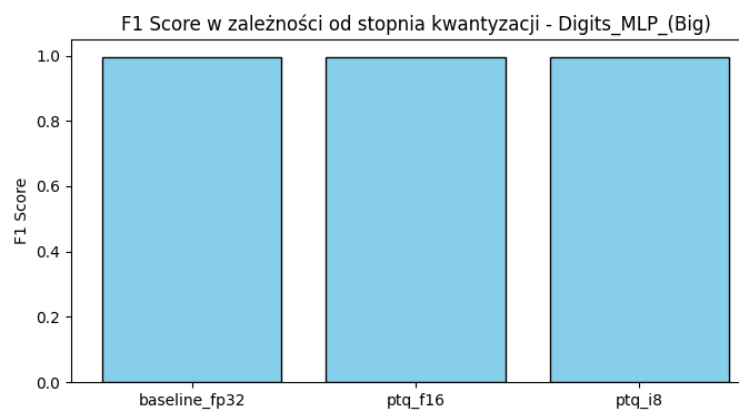
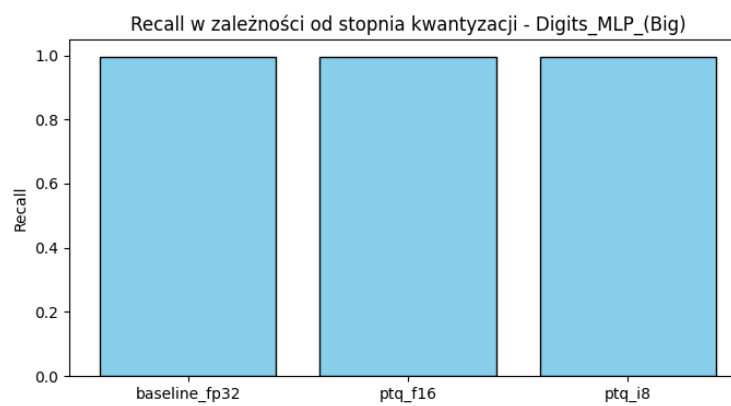
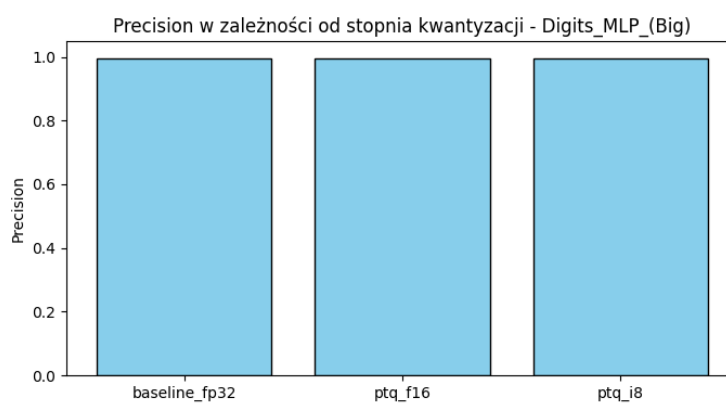
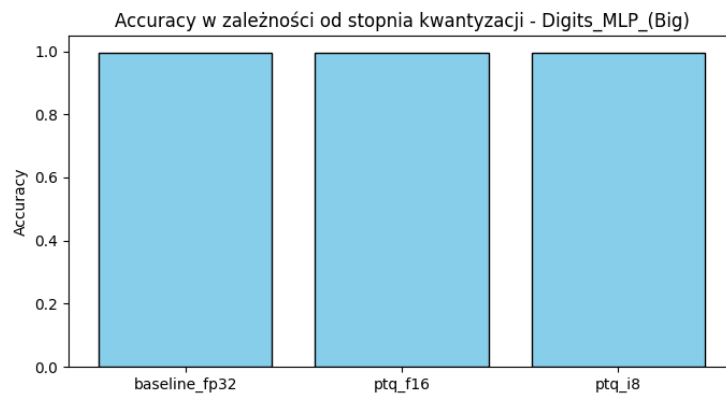
5.2. Osiągi sieci w zależności od stopnia kwantyzacji

Modele klasyfikujące, szczególnie większe modele, nie wykazują znacznego spadku dokładności oraz pozostałych metryk w wyniku kwantyzacji. Nawet może wystąpić sytuacja, gdzie kwantyzacja modelu do INT8 poprawi ostateczny wynik.

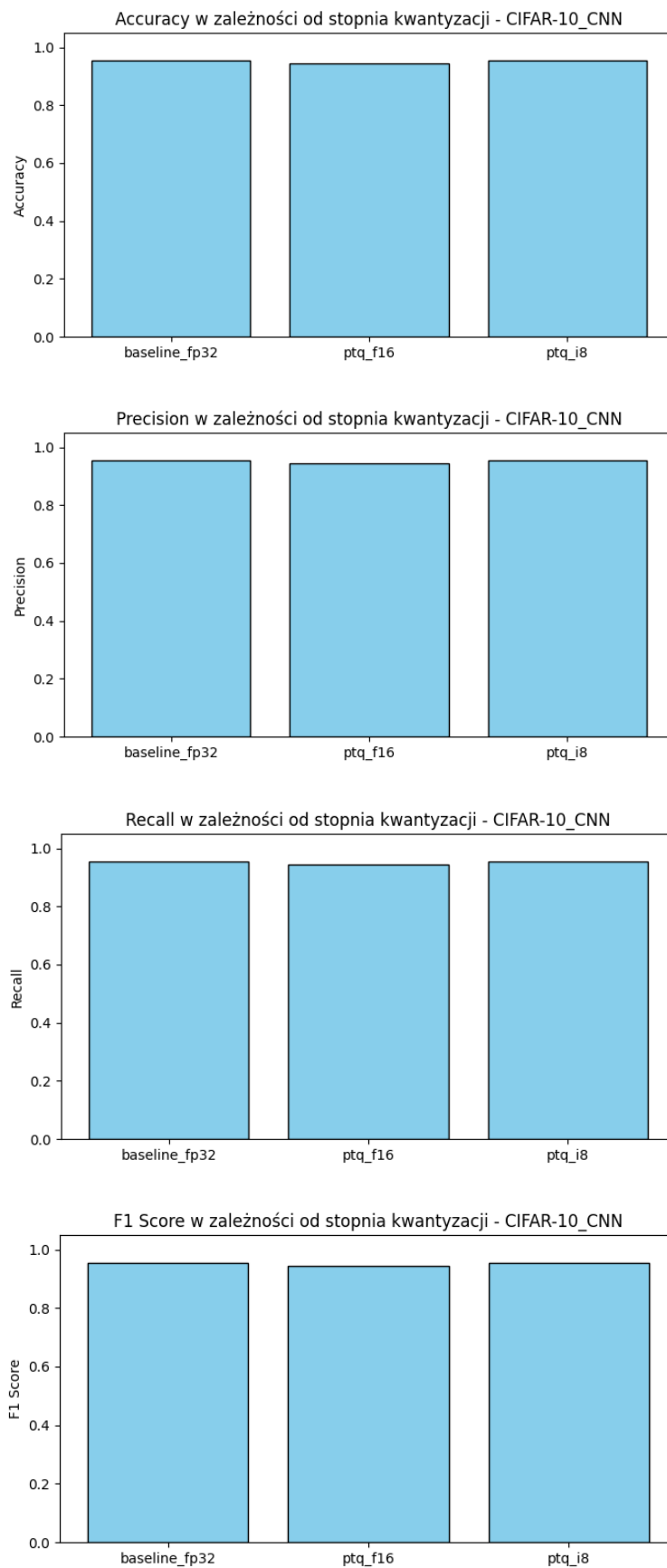
5.2.1. Mały MLP



5.2.2. Duży MLP

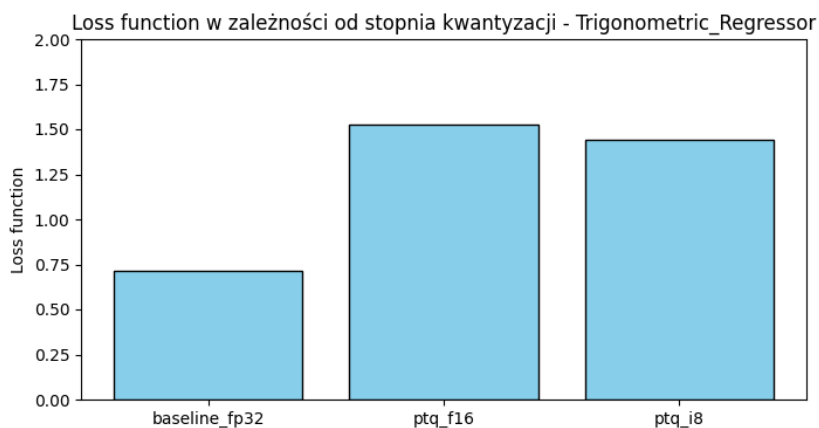


5.2.3. CNN



5.2.4 Regressor

Ponieważ ten model nie jest modelem klasyfikującym, nie można w jego przypadku użyć metryk wykorzystanych w poprzednich modelach. Jakość działania takiej sieci ocenia się bezpośrednio funkcją strat (MSE).



6. Wnioski

Na podstawie przeprowadzonych testów i zebranych metryk można sformułować następujące wnioski dotyczące wpływu kwantyzacji na sieci neuronowe:

Małe modele są wrażliwe na utratę precyzji (MLP mniejszy): W przypadku najmniejszej sieci odnotowano wyraźny spadek jakości. Model bazowy osiągał dokładność na poziomie 0.75, która po kwantyzacji spadła do około 0.62 dla FLOAT16 oraz 0.67 dla INT8. Wynika to z faktu, że małe modele mają bardzo ograniczoną pojemność informacyjną i brakuje w nich nadmiarowości parametrów. Obcięcie precyzji wagi drastycznie zaburza ich działanie, co sprawia, że kwantyzacja tak małych architektur jest nie tylko pozbawiona sensu pamięciowego, ale wręcz szkodliwa dla ich skuteczności.

Duże modele klasyfikujące świetnie znoszą kwantyzację (MLP większy, CNN): Dla większych modeli klasyfikujących spadek jakości po kwantyzacji był niezauważalny. Wariant FLOAT16 okazał się minimalnie gorszy od bazowego FLOAT32, natomiast model w formacie INT8 dorównywał modelowi bazowemu, a miejscami wypadał od niego nawet nieznacznie lepiej. Potwierdza to, że duże sieci są wysoce odporne na szum wprowadzany przez kwantyzację, a obniżenie precyzji może niekiedy działać jak dodatkowa regularyzacja modelu.

Modele regresyjne wymagają wysokiej precyzji (Regressor): W przeciwieństwie do klasyfikatorów, regresor odnotował natychmiastowy i znaczny spadek jakości działania. Dla formatu FLOAT16 błąd był dwukrotnie większy niż w modelu bazowym (wersja INT8 poradziła sobie nieznacznie lepiej niż FLOAT16, ale wciąż gorzej od FLOAT32). Przyczyną tego zjawiska jest natura problemu: klasyfikatory wskazują jedynie najbardziej prawdopodobną klasę (wymagają jedynie zachowania relacji większości między wynikami), natomiast modele regresyjne muszą wiernie odtwarzać ciągłą funkcję matematyczną. Utrata precyzji wag bezpośrednio i negatywnie przekłada się na dokładność wyliczanych wartości ciągłych.

Podsumowanie końcowe: Kwantyzacja wag jest niezwykle skuteczną metodą optymalizacji (zmniejszania rozmiaru i przyspieszania inferencji) dla głębokich i dużych modeli klasyfikacyjnych. Nie należy jej jednak stosować dla modeli bardzo małych oraz dla sieci realizujących zadania regresji, gdzie dokładność zmiennoprzecinkowa jest kluczowa dla poprawnego działania.